

Programming Assignment 4

Due at the beginning of your discussion session on
September 22-24, 2026

Reading

In addition to the following topics, the quiz syllabus includes any material covered in the lectures:

- In chapter 5 of Code Complete:
 - Section 5.1
 - Figure 5-2
 - “Form Consistent Abstractions”
 - “Encapsulate Implementation Details”
 - “Hide Secrets (Information Hiding)”
 - “Keep Coupling Loose”
- Section 19.6 in Code Complete
- Item 64 in Effective Java

Special Grading Guidelines

Starting with this assignment, an automatic C (or less) is triggered by:

- Any routine with complexity greater than 4, or by
- Any piece of code that is essentially repeated.

However, a submission that avoids these problems does not necessarily qualify for high quality craftsmanship.



Programming

First, make all the changes discussed in your discussion section. Additionally, you should refactor your code to make sure that it adopts the principles covered in the reading assignments.

In this assignment, you will implement a block representation of matrixes in quadrants and use it in Strassen's algorithm.

Quadrants

A set of quadrants is a specific block decomposition in which a matrix is viewed as four equal-size submatrices. For example, the matrix

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 6 & 7 & 8 & 9 \\ 5 & 4 & 3 & 2 \end{pmatrix}$$

can be viewed as the four quadrants

$$A = \begin{pmatrix} 1 & 2 \\ 5 & 6 \end{pmatrix}, B = \begin{pmatrix} 3 & 4 \\ 7 & 8 \end{pmatrix}, C = \begin{pmatrix} 6 & 7 \\ 5 & 4 \end{pmatrix}, D = \begin{pmatrix} 8 & 9 \\ 3 & 2 \end{pmatrix}$$

arranged as

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix}$$

Define a public method

```
Matrix::quadrant(  
    boolean horizontalOffset,  
    boolean verticalOffset)
```

that returns a sub-matrix corresponding to the offsets:

horizontalOffset	verticalOffset	sub-matrix
false	false	<i>A</i>
false	true	<i>C</i>
true	false	<i>B</i>
true	true	<i>D</i>

The purpose of `Matrix::quadrants` is to break a matrix in its quadrants. In the reverse direction, quadrants can be pieced together in a `Matrix` by defining a public final class `Quadrants` that contains the private final fields:

- `EntryMap<Matrix> quadrants` that represents the four blocks
- `int size` that represents the size of the underlying matrix and as a public getter.

The values of these fields are set by a private constructor. To handle size errors, add to the `InconsistentSizeException` a private final `Optional<Coordinates> quadrant`, whose value is set by the constructor and has a getter. Also, add to the `InconsistentSizeException` a second `validate` method

```
public static void validate(
    int referenceSize,
    Matrix otherMatrix,
    Optional<Coordinates> quadrant)
```

The public static method

```
Quadrants from(Map<Coordinates, Matrix> quadrants)
```

validates the arguments by making sure that `quadrants` and its four expected components are not null. Furthermore, if the four components differ in size, it throws an `IllegalArgumentException` whose cause is an appropriate `InconsistentSizeException`.

The public method `Matrix toMatrix()` returns a new `Matrix` with the entries derived from the four blocks in the `quadrants`.

Strassen's Algorithm

Define a public method `Matrix times(Matrix other)` to implement Strassen's multiplication algorithm. First, the `other` size is validated. Then, if both sizes are one, the algorithm returns a 1x1 matrix whose value is the product of the entries in the two matrixes. The main algorithm works recursively. To multiply

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \cdot \begin{pmatrix} E & F \\ G & H \end{pmatrix} = \begin{pmatrix} I & J \\ K & L \end{pmatrix}$$

it defines

$$S_1 = A \cdot (F - H)$$

$$S_2 = (A + B) \cdot H$$

$$S_3 = (C + D) \cdot E$$

$$S_4 = D \cdot (G - E)$$

$$S_5 = (A + D) \cdot (E + H)$$

$$S_6=(B-D)\cdot(G+H)$$

$$S_7=(A-C)\cdot(E+F)$$

and then calculates the products as:

$$I=S_5+S_6+S_4-S_2$$

$$J=S_1+S_2$$

$$K=S_3+S_4$$

$$L=S_1-S_7-S_3+S_5$$

Implement the algorithm and test it on at least three pairs of matrixes.

General Considerations

These classes may contain as many auxiliary private and package-private methods as you see fit, and additional package-private helper classes may be defined. However, any modification or addition to public classes and methods must be approved by the instructors at the discussion board.

You should write JUnit tests to make sure that your primary methods work as intended. However, we will revisit testing later on in the course, so extensive testing is not yet recommended. If you have never used JUnit, the JUnit module on canvas lists resources to get you started. Similarly, your code should have a reasonable number of comments, but documentation is going to be the topic of a future assignment. As a general guideline at this stage of the course, comments and tests should be like those accepted in CSDS 132.

Submission

Submission instructions will be posted shortly.